
UNIT 4 FUNCTIONS AND FILE HANDLING IN PYTHON

Structure

- 4.0 Introduction
- 4.1 Objectives
- 4.2 Function definition and calling
- 4.3 Scope of Variable
- 4.4 Function arguments
- 4.5 Returning from a function
- 4.6 Function objects
- 4.7 Lambda / Anonymous Functions
- 4.8 File Operations
- 4.9 Summary
- 4.10 Solution to Check your Progress
- 4.11 References/Further Readings

4.0 INTRODUCTION

Python lets you organise tasks into more manageable units called functions. Functions make the code modular. Modularity is the process of decomposing a problem into a set of sub-problems to reduce its complexity. Functions also make the code reusable. File handling is another important aspect discussed in this unit. File handling includes creating, deleting, and manipulating files. You can handle files using Python. In this unit, you will learn how to write your own functions. This unit also covers Python file handling, including opening, reading, writing, and closing files, which enables you to manage and manipulate data stored in files.

4.1 OBJECTIVES

After completing this unit, you will be able to:

- Define and call functions in your code,
- Explain the scope of functions,
- Define function objects,
- Create lambda functions, and
- Use file handling operations.

4.2 FUNCTION DEFINITION AND CALLING

A function is a block of code that performs a specific task. As a program grows, its complexity increases. Hence, it becomes important to organise the program into more manageable units. Further, it makes the code reusable.

There are three types of functions in Python-

1. Built-in functions – these are the functions which are already defined in the language. Example `print()`, `max()`, `input()`, etc.
2. User-defined functions- these are the functions that can be created or defined by the users according to their needs.
3. Anonymous functions

4.2.1 Creating user-defined functions

You can define a function using the *def statement* and give it a suitable name by following identifier rules. The process of defining a function is called *function definition*.

Syntax of function definition

```
def function_name ( list_of_parameters ) :  
    Statement (s)
```

Where,

`function_name` is the name of the function. A set of parentheses must follow a function name. Any name can be given to a function following the rules of identifiers.

The `list_of_parameters` is an optional field used to pass values or inputs to a function. It can be none or a comma-separated list of variables.

`Statement(s)` is a set of statements or commands within the function. Each time the function is called, all statements execute. Together, these statements form a function's body.

Even if a function is defined, it can never be executed until it is called. Therefore, to use a function, you must call it using a *function call statement*. A function can be called by its name with a set of parentheses and an optional list of arguments.

Syntax of function calling:

```
function_name( list_of_arguments)
```

Example: A function to display HELLO WORLD

```
def first_function():          # function definition  
    print("HELLO WORLD...!!!")  
  
first_function()              # function calling  
  
===== RESTART: C:/Users/test/Desktop/python_programs/abc.py =====  
HELLO WORLD...!!!  
>>>
```

A function needs to be defined only once, but you can call it any number of times using calling statements. They can be called not only in the same program, but also in different programs. This will be further discussed in the next unit.

Another example shown below is a program having a function to calculate the factorial of a number.

Example: A function to display the factorial of a number.

```
def factorial():          # function definition
    fact=1
    n=int(input("Enter a number :"))
    for i in range(1,n+1):
        fact=fact*i
    print("Factorial of ",n," is ",fact)

factorial()              # function calling
```

```
===== RESTART: C:/Users/test/Desktop/python_programs/factorial.py =====
Enter a number :4
Factorial of 4 is 24
>>>
```

4.3 SCOPE OF VARIABLE

The part of a code in which a variable can be accessed is called *the scope of a variable*. A variable's scope can be global or local, depending on its availability region.

Global variables are variables created outside any function. Their scope is global, hence can be used anywhere. You can also create global variables within a function using the keyword *global*.

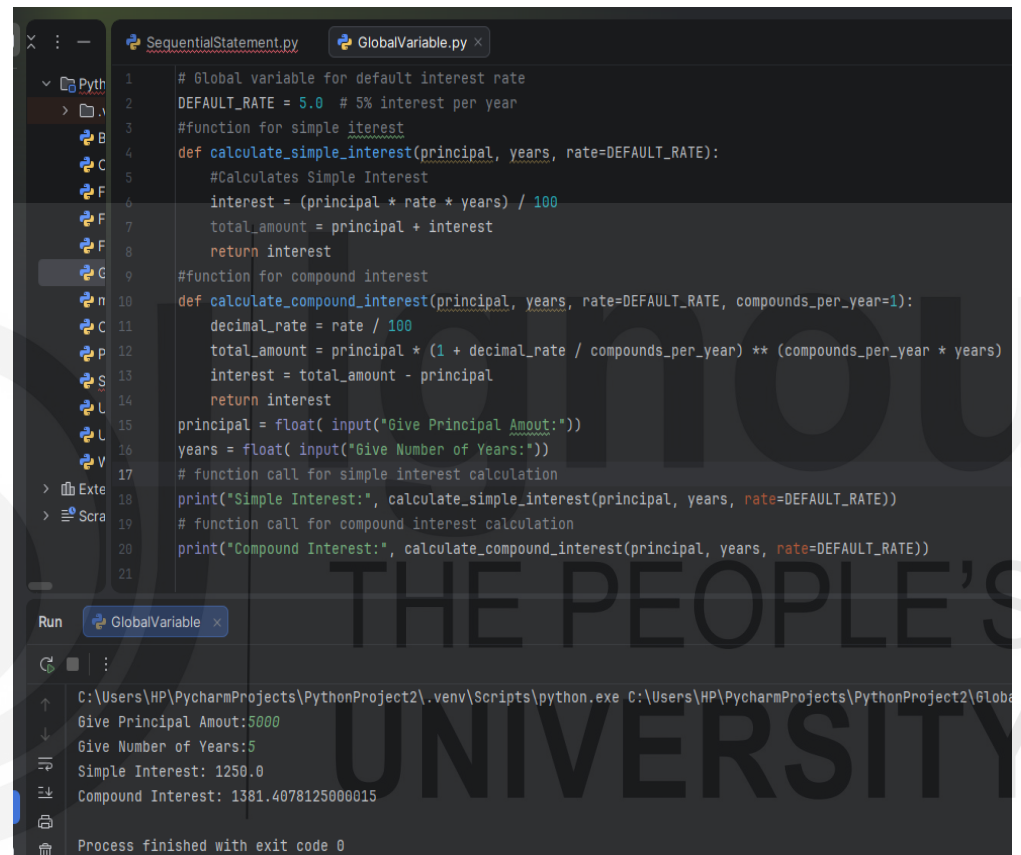
Local variables are variables created within a function's body or function's scope. You cannot access them outside the function. Their scope is limited to the function in which they are created.

Example: Use of local and global variables in interest calculation

```
# Global variable for default interest rate
DEFAULT_RATE = 5.0 # 5% interest per year
#function for simple interest
def calculate_simple_interest(principal, years,
rate=DEFAULT_RATE):
    #Calculates Simple Interest
    interest = (principal * rate * years) / 100
    total_amount = principal + interest
    return interest
#function for compound interest
def calculate_compound_interest(principal, years,
rate=DEFAULT_RATE, compounds_per_year=1):
    decimal_rate = rate / 100
    total_amount = principal * (1 + decimal_rate / compounds_per_year)
    ** (compounds_per_year * years)
    interest = total_amount - principal
    return interest
```

```
principal = float( input("Give Principal Amount:"))
years = float( input("Give Number of Years:"))
# function call for simple interest calculation
print("Simple Interest:", calculate_simple_interest(principal, years,
rate=DEFAULT_RATE))
# function call for compound interest calculation
print("Compound Interest:", calculate_compound_interest(principal,
years, rate=DEFAULT_RATE))
```

Execution of Screenshot of above Program:



The screenshot shows a Python IDE with two files: `SequentialStatement.py` and `GlobalVariable.py`. The code in `GlobalVariable.py` defines a global variable `DEFAULT_RATE = 5.0` and two functions: `calculate_simple_interest` and `calculate_compound_interest`. It then calls these functions with user input. The Run window shows the output: `Give Principal Amount:5000`, `Give Number of Years:5`, `Simple Interest: 1250.0`, and `Compound Interest: 1381.4078125000015`. The process finished with exit code 0.

In the above example, `DEFAULT_RATE` is global variable; hence, it can be used inside and outside any function. The variable `interest` is created inside the functions `calculate_simple_interest` and `calculate_compound_interest()`; it is local to them, and its scope is limited to these functions, so it cannot be used outside.

You can define global variables inside functions using *the global keyword*, as shown in the next example.

Example: Program to demonstrate creation of a global variable inside a function.

```
# G_Var is Global Variable
G_Var =45
def Fun_1():
```

```

L_Var =5
global G_Var
print("Global Variable:",G_Var)
G_Var = G_Var+10
print("Local Variable inside Function 1:", L_Var)
def Fun_2():
    # Here G_Var is a local variable and it will override the Global
    Variable
    G_Var= 15
    print("Local Variable inside Function 2 :", G_Var)
    G_Var +10
Fun_1()
Fun_2()
print("Global Variable:", G_Var)

```



The screenshot shows a Python IDE with a file named 'Global Local Test.py'. The code in the file is as follows:

```

1 # G_Var is Global Variable
2 G_Var =45
3 def Fun_1():
4     L_Var =5
5     global G_Var
6     print("Global Variable:",G_Var)
7     G_Var = G_Var+10
8     print("Local Variable inside Function 1:", L_Var)
9
10 def Fun_2():
11     # Here G_Var is a local variable and it will override the Global Variable
12     G_Var= 15
13     print("Local Variable inside Function 2 :", G_Var)
14     G_Var +10
15
16 Fun_1()
17 Fun_2()
18 print("Global Variable:", G_Var)

```

The output of the program is shown in the console:

```

Global Variable: 45
Local Variable inside Function 1: 5
Local Variable inside Function 2 : 15
Global Variable: 55
Process finished with exit code 0

```

If we create a variable inside a function having the same name as that of a global variable, then a separate variable of the same name gets created. In this case, the function accesses the local variable and hence cannot refer to the global variable, as shown in the example. In `Fun_1()`, the `global` keyword is used. In Python, we use the `global` keyword to modify a variable outside the current scope (usually inside a function) without creating a new local variable. In the above example *global G_Var* statement is for this purpose only.

Another example is given below where the global variable x is being modified inside the test() function.

```
x=1                                # global variable
def test():
    global x                        # to access global variable for modification
    print("inside function before modification x=",x)
    x=x+2
    print("inside function after modification x=",x)

test()
print("outside function x=",x)
```

===== RESTART: C:/Users/test/Desktop/python_programs/test.py =====

```
inside function before modification x= 1
inside function after modification x= 3
outside function x= 3
```

Example: Program showing use of Local and global variables of same name.

```
x=1                                # global variable
def test():
    x=2                            # local variable
    print("inside function x=",x)

test()
print("outside function x=",x)
```

===== RESTART: C:/Users/test/Desktop/python_programs/test.py =====

```
inside function x= 2
outside function x= 1
>>>
```

In Python, a function can access a global variable directly, but it cannot modify it unless you indicate that the variable is global. Then how can we modify global variables inside a function? The answer is the *global* keyword. It can be used not only to define/create global variables inside functions, but also to modify them. A very important point to note is that:

- i. You can use the global keyword to create a brand-new global variable from inside a function, even if that variable did not exist at the module level before.
- ii. By default, when you assign a value to a variable inside a function, Python creates a **local variable**. Therefore, if a function is not informed about a global variable properly using the global keyword, it will override the global variable with its local one.

The following program shows that you cannot modify a global variable inside a function. When you run this program, you will get the following error:

```

1 # x is Global Variable
2 x = 5
3 def Fun_1():
4     x = x + 2
5     print("Global Variable G_Var1:", x)
6 Fun_1()
7 print("Value of Global Variable G_Var1 at this stage:", x)

```

```

Run Global Access in Functionm x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe "C:\Users\HP\PycharmProjects\Python
Traceback (most recent call last):
  File "C:\Users\HP\PycharmProjects\PythonProject2\Global Access in Functionm.py", line 6, in <module>
    Fun_1()
  File "C:\Users\HP\PycharmProjects\PythonProject2\Global Access in Functionm.py", line 4, in Fun_1
    x = x + 2
    ^
UnboundLocalError: cannot access local variable 'x' where it is not associated with a value

Process finished with exit code 1

```

Let us see another example where a global variable is created inside a function named Fun_1().

```

1 # G_Var1 is Global Variable
2 G_Var1 = 45
3 Sum = 0
4 def Fun_1():
5     L_Var = 5
6     global G_Var1
7     print("Global Variable G_Var1:", G_Var1)
8     G_Var1 = G_Var1 + 10
9     global G_Var2 # G_Var2 is a global variable
10    G_Var2 = 10
11    G_Var2 = G_Var2 + 10
12    print("Local Variable inside Function 1:", L_Var)
13 def Fun_2():
14     # Here G_Var is a local variable and it will override the Global Variable
15     G_Var1 = 15
16     print("Local Variable inside Function 2 named G_var1:", G_Var1)
17 Fun_1()
18 Fun_2()
19 print("Value of Global Variable G_Var1 at this stage:", G_Var1)
20 Sum = G_Var1 + G_Var2
21 print("Sum of Global Variables G_Var1 and G_Var2:", Sum)

```

```

Run Global Local Test x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe "C:\Users\HP\PycharmProjects
Global Variable G_Var1: 45
Local Variable inside Function 1: 5
Local Variable inside Function 2 named G_var1: 15
Value of Global Variable G_Var1 at this stage: 55
Sum of Global Variables G_Var1 and G_Var2: 75

Process finished with exit code 0

```

Check your Progress – 1

1. What are the benefits of using functions? Also differentiate between function definition and function calling.

2. What is Scope of a variable? Explain local and global variables with example.

3. Write a function named *patter* to display the pattern given below:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

4.4 FUNCTION ARGUMENTS

Arguments are a way of passing values or input to a function. Arguments are passed to a function during function calls. Their values replace the function's parameters in the function definition.

Python provides various mechanisms for passing arguments to a function.

1. Required arguments
2. Positional arguments
3. Default arguments
4. Keyword arguments
5. Arbitrary arguments

1. Required Arguments

A required argument is any parameter without a default value. If you do not provide it when calling the function, Python raises a `TypeError`.

Example Program:

```
def Child(name, sex, age=18): # 'name' and 'sex' is required, 'age' is
    optional (has default 18)
    print(f"Hello {name}, you are a:, {sex}, you are {age}, years old.")
    Child("Rani", "Female") # Works! 'name' was supplied.
    Child( ) # TypeError: missing 1 required positional argument: 'name'
```

If you run the above program, you will get an error that :

TypeError: Child() missing 2 required positional arguments: 'name' and 'sex'
Because these two parameters are required parameters in the Child () function.

```

1 def Child(name, sex, age=18): # 'name' and 'sex' is required, 'age' is optional (has default 18)
2     print(f"Hello {name}, you are a: {sex}, you are {age}, years old.")
3     Child(name="Rani", sex="Female") # Works! 'name' was supplied.
4     Child() # TypeError: missing 1 required positional argument: 'name'
5
Run FunctionArguments x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\
Hello Rani, you are a: Female, you are 18, years old.
Traceback (most recent call last):
  File "C:\Users\HP\PycharmProjects\PythonProject2\FunctionArguments.py", line 4, in <module>
    Child() # TypeError: missing 1 required positional argument: 'name'
    ^^^^^^^
TypeError: Child() missing 2 required positional arguments: 'name' and 'sex'

Process finished with exit code 1

```

Now let us run the above program by modifying it as:

```

def Child(name, sex, age=18): # 'name' and 'sex' is required; 'age' is
optional (has default 18)
    print(f"Hello {name}, you are, {sex}, you are {age}, years old.")
Child("Rani", "Female") # Works! 'name' was supplied.
Child("Ravi", "Male", 23) # TypeError: missing 1 required positional
argument: 'name'

```

Then you will get output:

```

1 def Child(name, sex, age=18): # 'name' and 'sex' is required, 'age' is optional (has default 18)
2     print(f"Hello {name}, you are, {sex}, you are {age}, years old.")
3     Child(name="Rani", sex="Female") # Works! 'name' was supplied.
4     Child(name="Ravi", sex="Male", age=23) # TypeError: missing 1 required positional argument: 'name'
5
Run FunctionArguments x
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\Fun
Hello Rani, you are, Female, you are 18, years old.
Hello Ravi, you are, Male, you are 23, years old.
Process finished with exit code 0

```

In required arguments, the number of arguments should match the number of function parameters and should be given in the same order. If the number of arguments is not the same, an error will occur; also, if the correct order is not followed, the output can be incorrect.

2. Positional Arguments

A positional argument is assigned based on its **order/position** in the function call.

Example Program: Passing arguments to a function.

```
def calculator(a,b):
    print("Addition      :",a+b)
    print("Subtraction   :",a-b)
    print("Multiplication:",a*b)
    print("Division       :",a/b)

calculator(5,2)
```

```
===== RESTART: C:/Users/test/Desktop/python_programs/arguments.py =====
Addition      : 7
Subtraction   : 3
Multiplication: 10
Division      : 2.5
>>>
```

3. Default arguments

Default values of variables can be given in the parameters itself. So, if an argument is not given (passed) at the time of calling, it is replaced by the default value. Default values can be given for any number of arguments. Ensure that default arguments follow non-default arguments.

Example program to demonstrate use of default arguments:

```
def simple_interest(p,r=2,t=5):
    interest=(p*r*t)/100
    print(" Simple Interest is:",interest)

simple_interest(1000)      # p=1000, r=2 (default) ,t=5(default)
simple_interest(1000,5)    # p=1000, r=5 ,t=5(default)
simple_interest(1000,5,10) # p=1000, r=2 ,t=5
```

```
===== RESTART: C:\Users\test\Desktop\python_programs\arguments.py =====
Simple Interest is: 100.0
Simple Interest is: 250.0
Simple Interest is: 500.0
>>>
```

4. Keyword Arguments

As shown above, arguments must be given in the order in which parameters are defined; otherwise, the result will be affected. Keyword arguments are one way by which we can give arguments in any order. You specify the parameter name with each argument during the function call. In this case, position does not matter, but name does matter; hence, they are also called named arguments.

Example to demonstrate use of keyword arguments

```
def compound_interest(p,r,t):
    amount = p*pow(1+ (r/100),t)
    interest = amount-p
    print("Compound Interest is:",interest)

compound_interest(r=13,p=100000,t=5)    # passing keyword arguments
compound_interest(100000,13,5)         # passing positional arguments

===== RESTART: C:/Users/test/Desktop/python_programs/compound_interest.py =====
Compound Interest is: 84243.51792999991
Compound Interest is: 84243.51792999991
>>>
```

5. Arbitrary Arguments

Sometimes you need to pass a different number of arguments to the same function, or you don't know at function definition time how many arguments will be used when calling the function. Python provides the feature of arbitrary arguments to address this issue. Arbitrary arguments can be declared using the * symbol.

Example to demonstrate use of arbitrary arguments

```
def sum(*a):                # declaration of arbitrary argument
    sum=0
    for i in a:
        sum=sum+i
    print("Sum of Series",a," is:",sum)

sum(1,2,5,7,8,10,2)        # passing arbitrary arguments
sum(10,20,25)

===== RESTART: C:/Users/test/Desktop/python_programs/Arbitrary.py =====
Sum of Series (1, 2, 5, 7, 8, 10, 2) is: 35
Sum of Series (10, 20, 25) is: 55
>>>
```

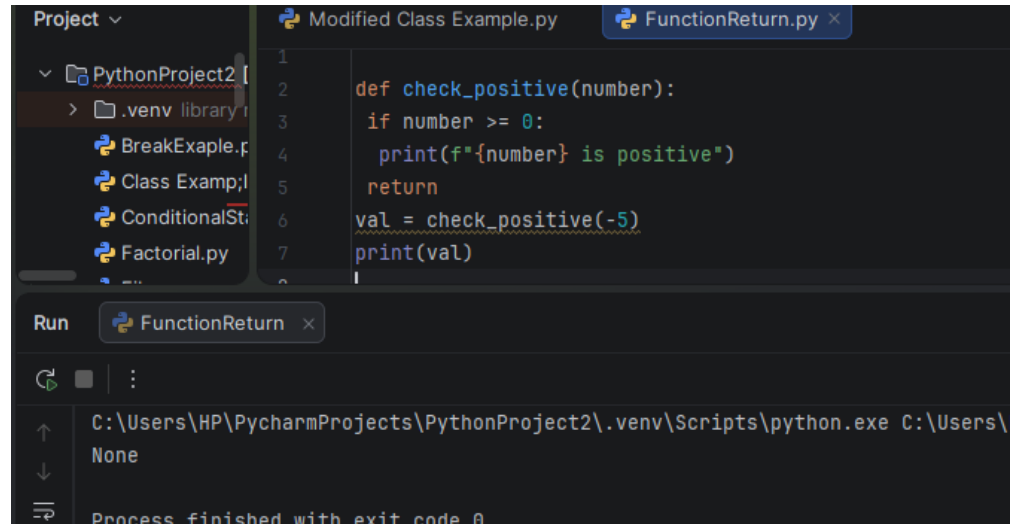
4.5 RETURNING FROM A FUNCTION

As we can pass values as input to function parameters, we can also return values from a function. You can do this using a **return statement**. By default, when we do not have any return statement in a function, the value **None** is returned from the function.

Syntax of returning from a function

```
def function_name ( arg1,arg2.... ) :
    .....
    .....
    return value
```

Function in below example program return None.



```
1
2 def check_positive(number):
3     if number >= 0:
4         print(f"{number} is positive")
5     return
6 val = check_positive(-5)
7 print(val)
```

Run FunctionReturn x

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\None

Process finished with exit code 0

In the example given below, the function example () returns the average value.

```
def example(a,b,c): # function with return statement
    avg=(a+b+c)/3
    return avg

result=example(2,3,4) # returned value passed to result variable
print("AVG OF NUMBERS:",result) # display return value by print statement
```

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====

AVG OF NUMBERS: 3.0

>>>

Although a function can have only one return statement, it does not mean that we can return only one value. It means the return statement can return only one object, and an object in Python can contain a single variable or multiple values (like a list, tuple, etc.). Hence, we can return multiple values with a single return statement. See the example below, where a function returns more than one value (one is sum and the other is avg) using a single return statement.

```
def example(a,b,c):
    sum=a+b+c
    avg=sum/3
    return sum,avg # function returning two values with single statement

result1,result2=example(2,3,4)
print("SUM OF NUMBERS:",result1)
print("AVG OF NUMBERS:",result2)
```

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====

SUM OF NUMBERS: 9

AVG OF NUMBERS: 3.0

>>>

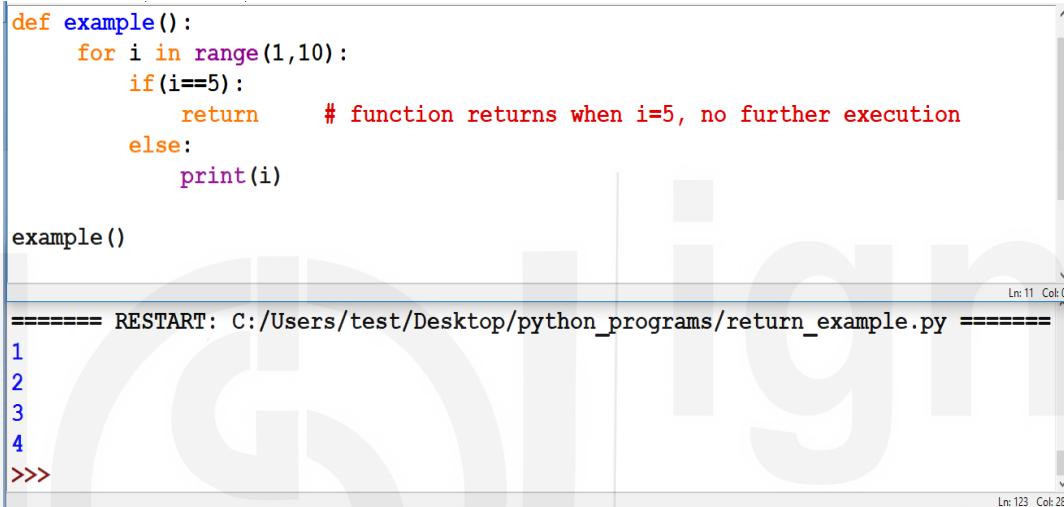
Benefits of using a return statement :

1. Values returned by the function can be used again in the rest of the program.

2. They can also be passed as arguments to other functions, providing better flow of control.
3. They can also be used to break out of a function in the middle.

When the return statement is executed(occurs) in the middle of statements in a function, then the statements following the return statement will never be executed. In the following example program, inside a function, there is a loop ranging from 1 to 9, but it will only be executed 4 times, since the function will return when the value of i reaches 5; hence, the remaining iterations will be skipped.

Program to demonstrate that a function returning in between the loop:



```
def example():
    for i in range(1,10):
        if(i==5):
            return # function returns when i=5, no further execution
        else:
            print(i)

example()

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====
1
2
3
4
>>>
```

4.6 FUNCTION OBJECTS

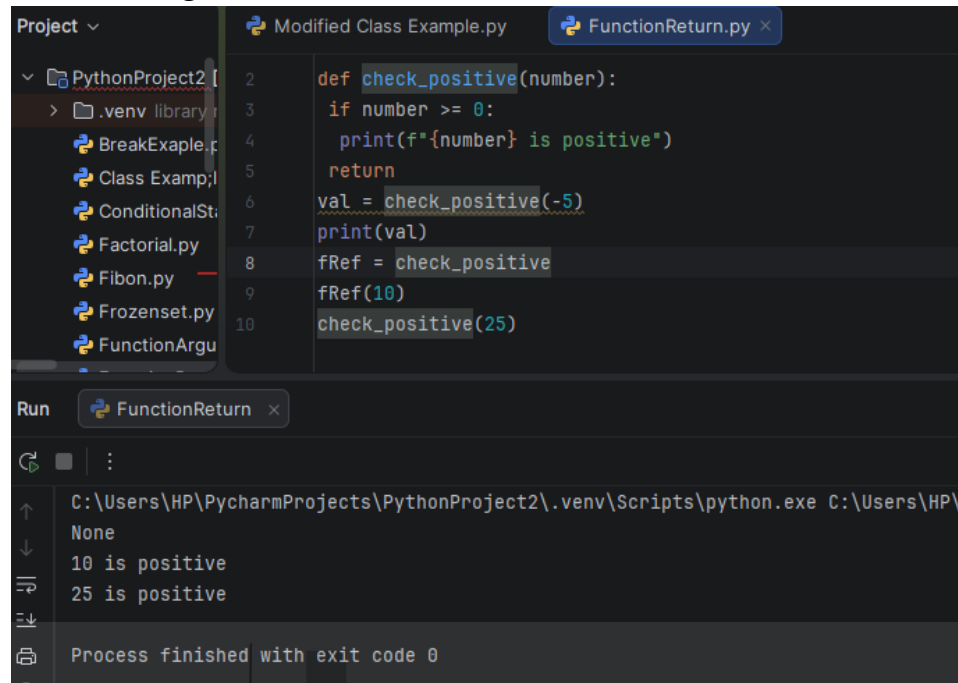
In Python, data is represented as objects. In fact, every piece of data—from numbers and strings to functions, classes, and modules – is an object in memory. Like Lists, Strings Sets etc, functions are also treated as objects. Functions in Python are first-class objects. First-class objects mean they are treated just like any other piece of data (like an integer, string, or list). Because functions are objects, they offer various features additional to the existing ones. These are:

1. Functions can be assigned to a variable
2. Functions can be used as elements of data structures
3. Functions can be sent as arguments to another function
4. Functions can be nested

1. Functions assigned to a variable

As functions are objects, they can be assigned to a variable (object). So you can call the function using the variable and parentheses. Assigning a function to a variable creates a reference to the same function or a function with two names. If you delete one function or variable, you can still reference the function by another name. The following example shows a function assigned to a variable.

Function assigned to variable:



The screenshot shows the PyCharm IDE with a project named 'PythonProject2'. The file explorer on the left lists several files, including 'FunctionReturn.py'. The main editor window displays the code in 'FunctionReturn.py':

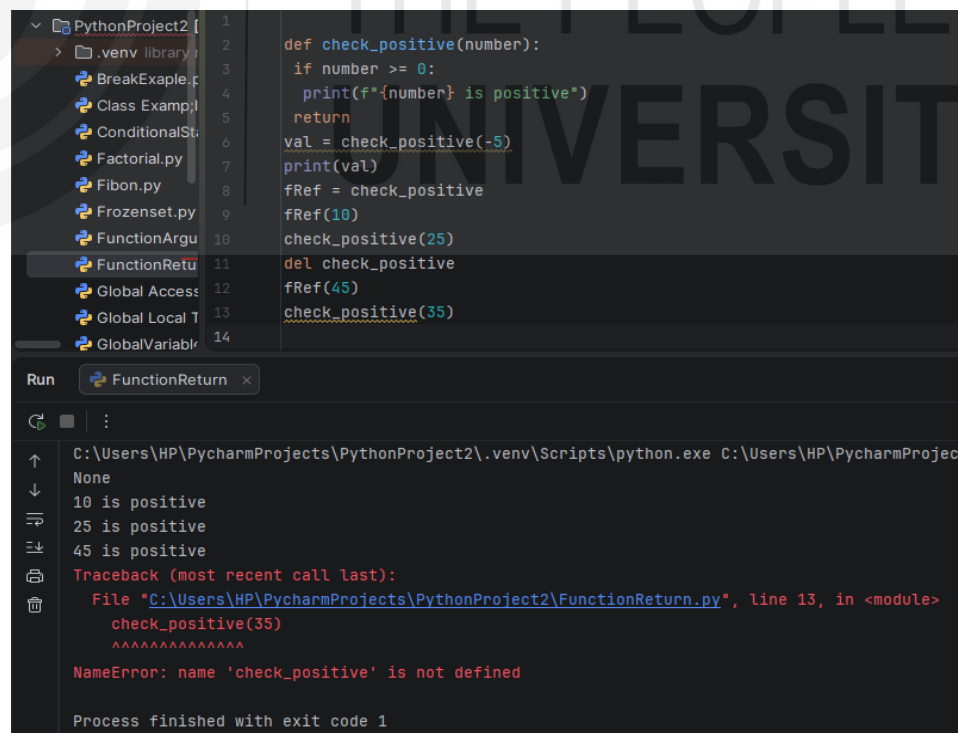
```
1 def check_positive(number):
2     if number >= 0:
3         print(f"{number} is positive")
4     return
5
6 val = check_positive(-5)
7 print(val)
8 fRef = check_positive
9 fRef(10)
10 check_positive(25)
```

The Run window at the bottom shows the execution output:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\FunctionReturn.py
None
10 is positive
25 is positive
Process finished with exit code 0
```

In the following code, you can clearly see that after deleting a function, you cannot access it by the same name. However, you can still call it using another name, as shown in the example program.

Deleting a reference to a function: The function call `check_positive(35)` can execute and generate an error because the reference to this function is deleted before it is called.



The screenshot shows the PyCharm IDE with the same project. The file explorer now includes 'FunctionReturn.py'. The main editor window displays the code in 'FunctionReturn.py':

```
1 def check_positive(number):
2     if number >= 0:
3         print(f"{number} is positive")
4     return
5
6 val = check_positive(-5)
7 print(val)
8 fRef = check_positive
9 fRef(10)
10 check_positive(25)
11 del check_positive
12 fRef(45)
13 check_positive(35)
14
```

The Run window at the bottom shows the execution output:

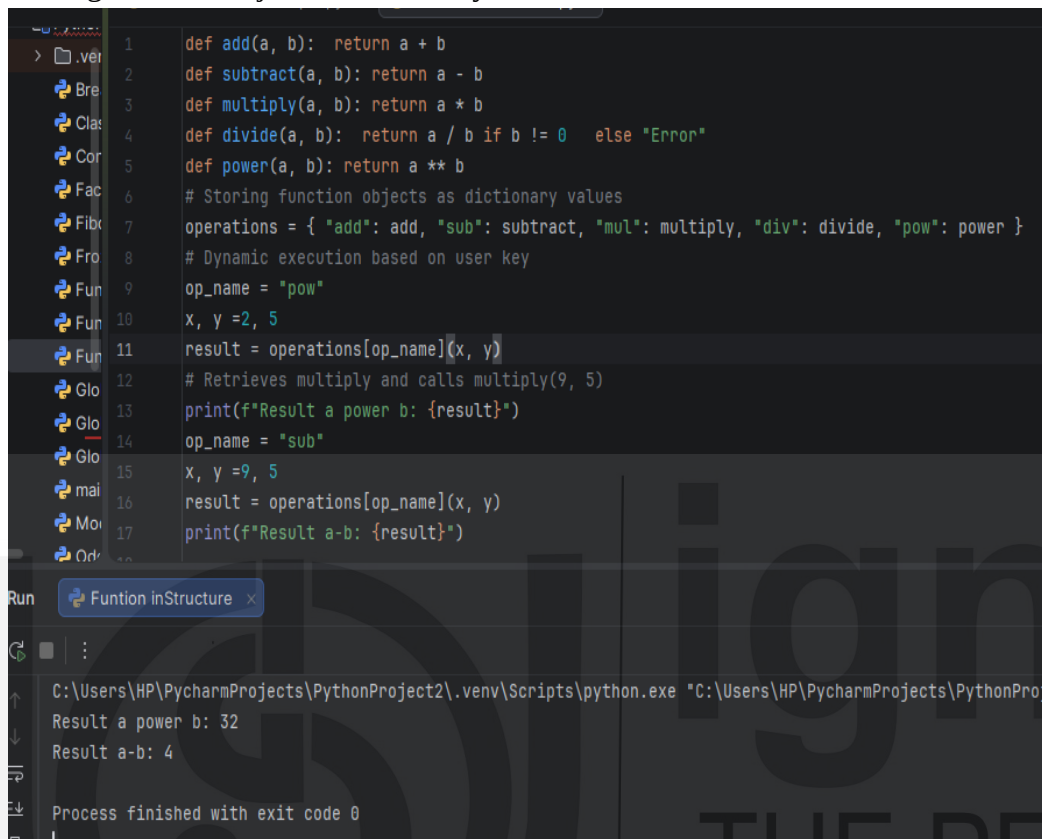
```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\FunctionReturn.py
None
10 is positive
25 is positive
45 is positive
Traceback (most recent call last):
  File "C:\Users\HP\PycharmProjects\PythonProject2\FunctionReturn.py", line 13, in <module>
    check_positive(35)
    ^^^^^^^^^^^^^^^^^
NameError: name 'check_positive' is not defined
Process finished with exit code 1
```

2. Functions as elements of a data structure

Functions can be passed as elements in any data structure. This is useful when we want to apply different functions to the same input. Storing functions as

elements inside data structures (lists, dictionaries, tuples, sets, etc.) is a powerful technique in Python. Below is an example. We use various built-in functions to demonstrate this.

Storing function objects as dictionary values:



```
1 def add(a, b): return a + b
2 def subtract(a, b): return a - b
3 def multiply(a, b): return a * b
4 def divide(a, b): return a / b if b != 0 else "Error"
5 def power(a, b): return a ** b
6 # Storing function objects as dictionary values
7 operations = { "add": add, "sub": subtract, "mul": multiply, "div": divide, "pow": power }
8 # Dynamic execution based on user key
9 op_name = "pow"
10 x, y = 2, 5
11 result = operations[op_name](x, y)
12 # Retrieves multiply and calls multiply(9, 5)
13 print(f"Result a power b: {result}")
14 op_name = "sub"
15 x, y = 9, 5
16 result = operations[op_name](x, y)
17 print(f"Result a-b: {result}")
```

Run Function in Structure

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe "C:\Users\HP\PycharmProjects\PythonProject2\return_example.py"

Result a power b: 32
Result a-b: 4

Process finished with exit code 0

3. Function as argument to another function

You can pass objects as arguments to a function. Since functions are objects in Python, they can also be passed as arguments to functions. This technique allows you to apply multiple operations to a particular set of inputs. Given below is an example of a function calling another function as an argument, where the function named `add` is passed as an argument to the function `display`.

Example to show a function calling another function as an argument :

```
def add(n):
    sum=0
    for i in n:
        sum=sum+i
    return sum

def display(l,func):
    print("Addition of elements:",func(l))

display([1,2,3,4,5,6,7],add)    # function display() calling function add()
```

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====

Addition of elements: 28

>>>

4. Nested function

Functions created within another function are called nested functions. As in Python, functions are objects. The inner function, which is created within some outer function, has access to all the variables of the enclosing scope. Inner functions can never be directly called outside the outer function. Hence, it provides a security feature called Encapsulation. Inner functions are hidden from the global scope (provide encapsulation). They can only be called from within the enclosing function.

Example: Nested function

```
def outer(text):          # outer function
    print(str.upper(text))

    def inner():          # nested or inner function
        print(str.lower(text))

    inner()               # nested function can only be called inside outer function

outer("Nested Function") # call to outer function makes indirect call to inner
inner("Nested Function") # shows error: nested function cannot be called outside
```

===== RESTART: C:/Users/test/Desktop/python_programs/return_example.py =====

```
NESTED FUNCTION
nested function
Traceback (most recent call last):
  File "C:/Users/test/Desktop/python_programs/return_example.py", line 12, in <module>
    inner("Nested Function") # shows error: nested function cannot be called outside
NameError: name 'inner' is not defined
>>>
```

In the above program, as the inner function's name is bound only within the local namespace of the outer function, calling it directly from the outside results in a NameError:

You should use an inner function when a function requires a small helper routine that is only useful inside that specific function; defining it as an inner function keeps it hidden from the rest of your program. Also, if a multi-step function repeats a 3-to-4 line operation multiple times internally, you can define an inner function that prevents duplicating code without exposing the operation globally.

4.7 LAMBDA / ANONYMOUS FUNCTIONS

Python allows you to create unnamed functions called anonymous *functions*. They are not declared with the *def* keyword; the *lambda* keyword is used to create these functions. Hence, these functions are also called lambda functions. Like other functions, these functions can also contain arguments, but there can be *only one statement* in the body of these functions. The statement is evaluated, and the result is returned. When you need a function for a short operation (similar to macros in the C language) or only for a short time, lambda functions are best suited.

```
Variable = lambda arg : statement
```

Here, arg is one or more arguments passed to a function, and statement consists of the operation performed by a function.

Below is an example in which two lambda functions are created. The first function calculates the cube of a given argument, and the second function adds two given arguments. The lambda function is then assigned to a variable (a function assigned to a variable, discussed in the previous section), which can then be used to call the function with arguments and parentheses.

Example to show use of Lambda functions:



The screenshot shows a Python IDE with three tabs: 'Modified Class Example.py', 'Funtion inStructure.py', and 'LambdaFn.py'. The 'LambdaFn.py' tab is active, displaying the following code:

```
1 divide = lambda a, b : a / b if b != 0 else "Error"
2 power = lambda a, b : a ** b
3 print("a/b:", divide(a: 20, b: 5))
4 print("a power b:", power(a: 3, b: 4))
5
```

Below the code editor, the 'Run' console shows the output of the program:

```
a/b: 4.0
a power b: 81
Process finished with exit code 0
```

Lambda functions are mostly used as arguments to other higher-order functions like `map()` and `filter()`.

map() function: `map()` is a built-in function which takes two arguments; the first argument given is a function (can be a lambda function), and the second argument is given as a list. The `map()` function returns a list of elements obtained by applying the given function to each element of a list.

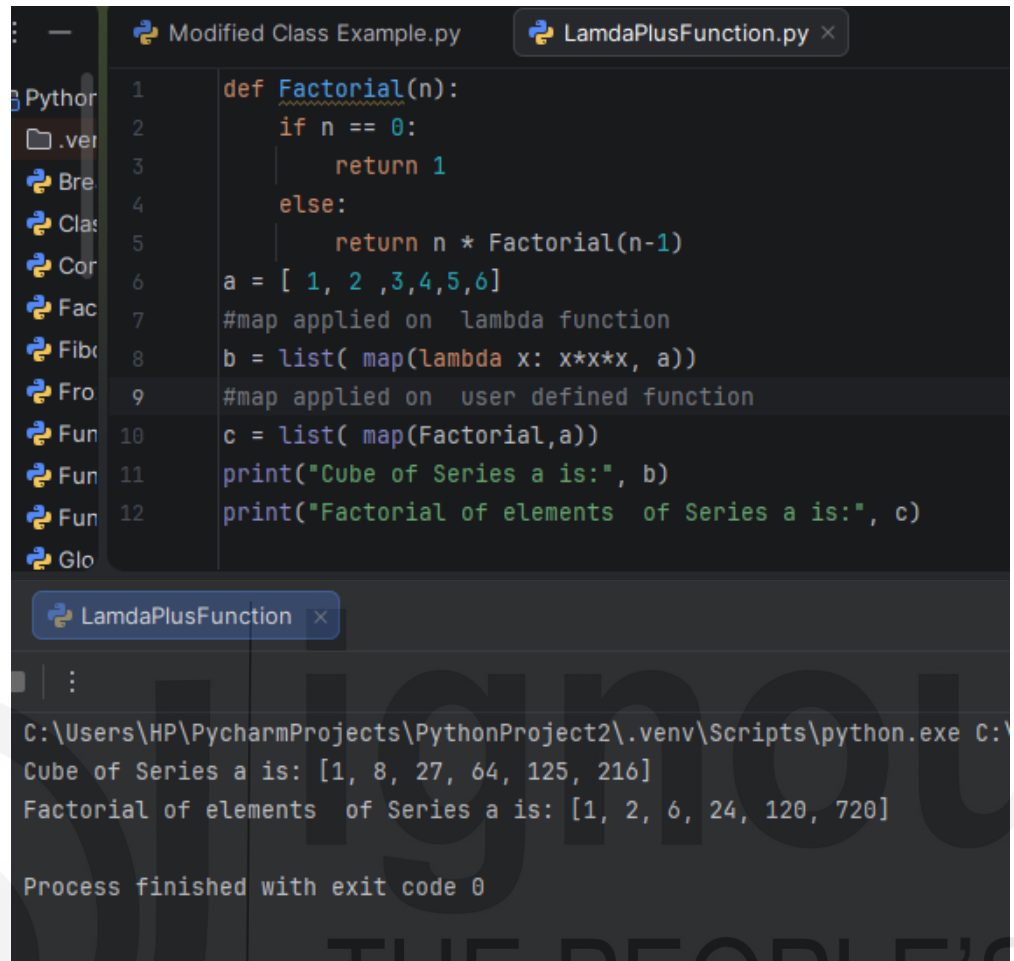
filter() function: `filter()` is another built-in function which takes two arguments. The first argument is a function, and the second is a list of elements, just as the `map()` function. Here, the given function is applied to each of the elements of the list and returns a list of items for which the function evaluates to True.

Syntax of map and filter functions

```
map ( function , [ list ] )
```

```
filter ( function , [ list ] )
```

Example of application of map() function with lambda function and user-defined function:

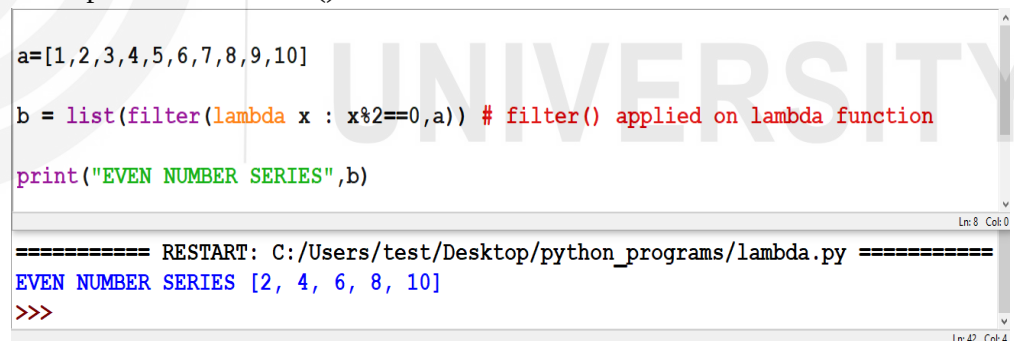


```
1 def Factorial(n):
2     if n == 0:
3         return 1
4     else:
5         return n * Factorial(n-1)
6 a = [ 1, 2 ,3,4,5,6]
7 #map applied on lambda function
8 b = list( map(lambda x: x*x*x, a))
9 #map applied on user defined function
10 c = list( map(Factorial,a))
11 print("Cube of Series a is:", b)
12 print("Factorial of elements of Series a is:", c)
```

Output:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\PycharmProjects\PythonProject2\lambda_plus_function.py
Cube of Series a is: [1, 8, 27, 64, 125, 216]
Factorial of elements of Series a is: [1, 2, 6, 24, 120, 720]
Process finished with exit code 0
```

Example of use of filter() function with lambda function



```
a=[1,2,3,4,5,6,7,8,9,10]
b = list(filter(lambda x : x%2==0,a)) # filter() applied on lambda function
print("EVEN NUMBER SERIES",b)
```

Output:

```
===== RESTART: C:/Users/test/Desktop/python_programs/lambda.py =====
EVEN NUMBER SERIES [2, 4, 6, 8, 10]
>>>
```

Check your Progress – 2

1. Write a function multiply_all(*numbers) which multiplies all the arguments and returns its value.

.....

.....

.....

.....

2. What are the various ways of passing arguments to a function?

.....

.....

.....

.....

3. Write a function which takes a list of numbers as an argument and returns a list of unique elements from it.

.....

.....

.....

.....

4. Write a program to find the cube of numbers in a list using a lambda function.

.....

.....

.....

.....

5. Write a program to add two given lists using the map function.

.....

.....

.....

.....

6. Write a program to display vowels from a given list of characters using the filter function.

.....

.....

.....

.....

4.8 FILE OPERATIONS

Main memory is volatile; hence, nothing can be stored permanently in it. To store the information permanently, files are used. You can permanently store data in non-volatile memory (e.g. hard disk, pen drive) using a file.

A file is a named location on disk to store related information. File handling is an important feature provided by all languages to address this need. You can take input from files instead of from users; you can also save output permanently in files for later access. You can append and update a file. All these functionalities come under file handling. Like other languages, Python also provides various built-in functions for file handling operations.

In Python, when you are programming, a “file” refers to a dedicated Python file object. This object is used to interact with data stored on your hard drive (like .txt, .csv, or other files). File handling in Python involves opening, reading, writing, and closing files, enabling the management and manipulation of data stored in files.

Python supports two types of files: binary and text.

Binary files – These files can be represented as 0's and 1's. As you know, everything in a computer is saved as 0s and 1s. Applications can process these files if they know the file's structure. Image files are an example of binary files. More examples of binary files include: .png, .pdf, .mp3, .exe, .dat. For these files, Python leaves the raw bytes as-is (bytes object). It does not try to parse them into letters or lines of text, because doing so would corrupt non-text data like images, compressed archives, or executables.

Text files – These files are organised as a sequence of characters. Here, each line is separated by a special end-of-line character. Some examples of text files are: .txt, .py, .csv, .log . Python interprets the underlying bytes as human-readable text using a character encoding such as: UTF-8 or ASCII. When you read them, Python converts the raw bytes into a string (str).

Any file handling operation can be performed in three steps-

1. Opening a file
2. Operating on the file – read, write, append, etc
3. Closing a file

Opening a file

A file can be opened in Python using the built-in function `open()`. By default, files are opened in read text ('r') file mode.

Syntax of `open()` function

```
file = open ( "file_name.ext ", "mode_of_operation")
```

Where,

`File_name.ext` - is the name of the file to be opened, and `ext` is the extension of the file.

`Mode_of_operation` - is the mode in which the file needs to be opened.

As a Python file is an object returned by the function. You can use this object for further operations on the file. The possible modes of operation in Python are:

Mode of opening file	Functionality
'r'	Opens a file for reading. It is the default mode
'w'	Opens a file for writing. Overwrites a file if it already exists. Creates a new file if it does not exist.
'a'	Opens a file for appending.
'r+'	Opens a file for both reading and writing.

'w+'	Same as 'r+' but creates a new file if it does not exist and overwrites if it exists
'a+'	Opens a file for appending and reading
'x'	Creates a new file. Fails if already exists. Added in Python 3.
'rb'	Opens a file for reading in binary mode.
'wb'	Same as 'w' except data is in binary
'ab'	Same as 'a' except data is in binary

Example of opening a file in different mode:

```

1  # 1. Relying on default behavior: file is opened in read text ('r') file mode |
2  with open("Test.txt") as file:
3      filecontent = file.read()
4      print(filecontent)
5  # 2. Explicitly setting read mode
6  with open("Test.txt", "r") as file:
7      filecontent = file.read()
8      print(filecontent)
9  # 3. Explicitly setting read + text mode
10 with open("Test.txt", "rt") as file:
11     filecontent = file.read()
12     print(filecontent)
13     file.close()
14     print("File is closed")

```

FileHandling x FileHandling x

C:\Users\DELL\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\DELL\PycharmF

Let us test the file operation
Let us test the file operation
Let us test the file operation
File is closed

Process finished with exit code 0

You can see in the above program that the file named “Test.txt” is opened, and here, because of its safety and readability, the with statement is used. Use of *with* is considered very good practice for file handling in modern Python. Various file-handling operations are performed.

Reading from a file

There are many ways to read from a file. The functions below are available for reading from a file. To read data from a file, it must exist. In the above program, the `filenameread()` function is used to read a file named “Test.txt” in the directory files within the present directory.

In the table below, it is assumed that file is the object returned from the open() function.

Function	Syntax	Description
read()	file.read ()	Returns entire file contents as a string
	file.read (n)	Returns n characters from beginning of file as string
readline()	file.readline()	Returns single line of file at a time. First line in this case.
readlines()	file.readlines()	Returns a list of all lines

Writing to a file

Similar to read operations, Python provides functions to write data to a file.

Function	Syntax	Description
write()	file.write(“text”)	Writes text or string to a file

Some more Operations on files

Python allows many other file operations. Listed below are some functions used for different file-handling operations.

Function	Syntax	Description
tell()	file.tell()	Returns the current cursor position in file
seek()	file.seek(pos)	Places the file cursor to the position pos
os.stat()	os.stat(filename)	This method is used to get the display status of the file given as an argument.
os.path.exists()	os.path.exists(‘arg’)	Returns true if the file or directory of name ‘arg’ exists
os.path.isfile()	os.path.isfile(‘arg’)	Returns true if the file of name ‘arg’ exists
os.path.isdir()	os.path.isdir(‘arg’)	Returns true if the directory of name ‘arg’ exists
shutil.copy()	shutil.copy(src,dst)	This function is part of Python’s built-in shutil (Shell Utilities) module. It copies a file from src to des file.
os.path.getsize()	os.path.getsize (filename)	Returns the size of the file

Closing a file

After completing all operations on a file, you must close it properly. This step frees up the resources and allows graceful termination of file operations.

```
file.close()
```

In Python, `import` is a keyword used to bring built-in or external code libraries (called modules) into your script. You will learn more about `import` in the coming units of this course. Let us look at an example program on Python File handling. In this program, we will *import* `os` and `time` (two essential Python Standard Library modules). These modules come built into Python, so you do not need to install them separately. The following Python program creates a file and writes a few lines in it. Then it reads the file's contents and displays them. It checks the file size, absolute path, and creation date and time. Then it appends more content to the file and rechecks the file size.

```
import os
import time
# Creating the file name = "My.txt"
print("---- 1. Creating FILE and WRITING few lines ----")
# Open file in write mode ('w')
with open("My.txt", "w", encoding="utf-8") as file:
    file.write("Line 1: Welcome to Python Programming.\n")
    file.write("Line 2: This course is dedicated to Python.\n")
    file.write("Line 3: The course code of this course is MCS-082.\n")
print(f"Successfully created and written to '{'My.txt'}'.\n")
print("---- 2. Reading FILE content ----")
# Now open file in read mode ('r')
with open("My.txt", "r", encoding="utf-8") as file:
    content = file.read()
    print("File Contents:")
    print(content)
print("---- 3. Checking FILE Size & modification details ----")
# 1. File existence check
if os.path.exists("My.txt"):
    print(f"File exists: Yes")
# 2. File size in bytes
file_size = os.path.getsize("My.txt")
print(f"File Size: {file_size} bytes")
# 3. Absolute path
abs_path = os.path.abspath("My.txt")
print(f"Absolute Path: {abs_path}")
# 4. Last modification time
mod_time = os.path.getmtime("My.txt")
readable_time = time.ctime(mod_time)
print(f"Last Modified: {readable_time}")
print("\n--- 4. Append more content to FILE ---")
# Open file in append mode ('a') to add lines without overwriting
with open("My.txt", "a", encoding="utf-8") as file:
    file.write("Line 4: This line we have appended later.\n")
# Re-checking size after append
new_size = os.path.getsize("My.txt")
print(f"Updated File Size: {new_size} bytes")
file.close()
```

After executing this file, you will get:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\
--- 1. Creating FILE and WRITING few lines ---
Successfully created and written to 'My.txt'.

--- 2. Reading FILE content ---
File Contents:
Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.

--- 3. Checking FILE Size & modification details ---
File exists: Yes
File Size: 137 bytes
Absolute Path: C:\Users\HP\PycharmProjects\PythonProject2\My.txt
Last Modified: Tue Aug 11 14:32:45 2026

--- 4. Append more content to FILE ---
Updated File Size: 180 bytes

Process finished with exit code 0
```

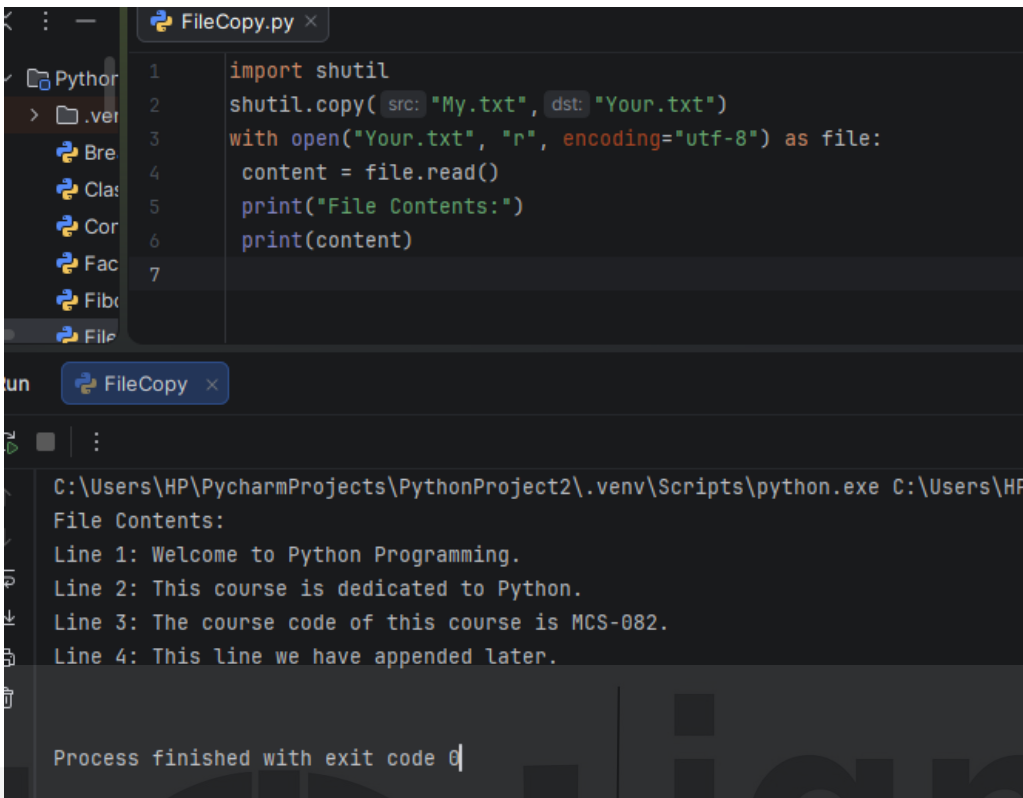
You can read the final content of the file and display the result as a practice exercise in the program above.

Note that the file we want to read or write may be in another directory or folder. In that case, we must provide the file's absolute or relative path along with its name. The path separated by \ must be preceded by one more \ (backslash). If the file to be read or written is in the same folder in which the Python program is, then we need not give its path.

Some file-handling methods may not be safe. An exception may occur during file operations, and the code may exit without closing the file. To make this easier, Python uses a statement called *with*, which ensures the file closes when the code within the *with* block exits. You don't need to call the `close()` function explicitly. You can see the use of *with* in the program given on the previous page.

Copying a file

To copy a file from one place to another, Python offers several methods. One such way of copying a file is by importing a module named 'shutil', which contains a built-in function to create a copy of a file. You can see in the following program where the file "My.txt", which we created in the previous example, is copied to a file named "Your.txt". You may also notice that the content of "Your.txt" matches the content we kept in "My.txt" after appending.



```

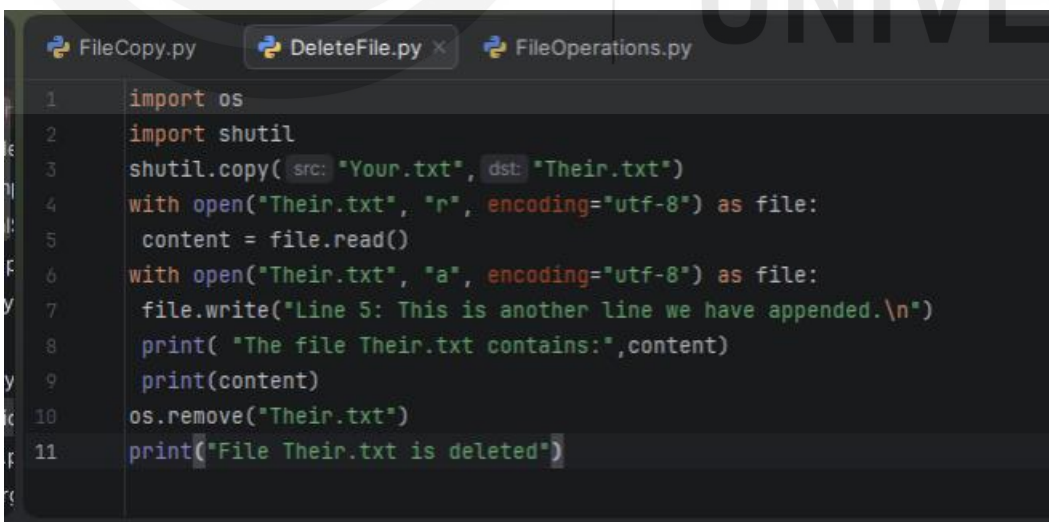
FileCopy.py
1  import shutil
2  shutil.copy(src="My.txt", dst="Your.txt")
3  with open("Your.txt", "r", encoding="utf-8") as file:
4      content = file.read()
5      print("File Contents:")
6      print(content)
7
run FileCopy
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
File Contents:
Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.
Line 4: This line we have appended later.

Process finished with exit code 0

```

Deleting a file or folder

A file can be deleted using the `remove()` function from the `os` module. Before using this function, you should move to the directory in which the file to be removed exists. Similarly, to delete a folder or directory, you can use the `os.rmdir()` function. The directory to be removed must be empty; otherwise an error will be shown. In the example given below, we want to delete a file named “Their.txt”. You can use the `os.listdir()` function to display the list of files present in a directory.



```

FileCopy.py DeleteFile.py FileOperations.py
1  import os
2  import shutil
3  shutil.copy(src="Your.txt", dst="Their.txt")
4  with open("Their.txt", "r", encoding="utf-8") as file:
5      content = file.read()
6  with open("Their.txt", "a", encoding="utf-8") as file:
7      file.write("Line 5: This is another line we have appended.\n")
8      print("The file Their.txt contains:", content)
9      print(content)
10 os.remove("Their.txt")
11 print("File Their.txt is deleted")

```

When you execute the above program, you will get:

```
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP
The file Their.txt contains: Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.
Line 4: This line we have appended later.

Line 1: Welcome to Python Programming.
Line 2: This course is dedicated to Python.
Line 3: The course code of this course is MCS-082.
Line 4: This line we have appended later.

File Their.txt is deleted

Process finished with exit code 0
```

Check your Progress – 3

1. Write a program to display the frequency of each word in a given file.

.....

.....

.....

.....

2. Write a program to display the first n lines from a given file, where n is given by the user.

.....

.....

.....

.....

3. Write a program to display the size of a file in bytes.

.....

.....

.....

.....

4.9 SUMMARY

This unit explores fundamental Python programming concepts, with greater emphasis on modular code design through functions. It covers how to define and call functions. Also, explained variable scope and the distinction between local and global visibility. Additional key topics include passing arguments to functions and writing concise Lambda expressions. Finally, the unit explains file handling in Python by demonstrating how to create, open, read, write, and safely manage files to ensure persistent program data.

4.10 SOLUTIONS TO CHECK YOUR PROGRESS

Answer to Check Your Progress-1

1. The various benefits of using functions are –

- i) It decomposes a larger problem into more manageable units and helps in eliminating repetitive code blocks
- ii) It allows reusability of code
- iii) Break complex problems into smaller, isolated, manageable chunks and reduce duplication
- iv) Hide complex underlying algorithms(implementation) behind a clear, descriptive function name and make the code self-documenting.
- v) It makes code easy to understand, use and debug

The process of defining a function is called *function definition*. It describes how a function works. It includes – function name, list of arguments and function body.

```
Syntax def function_name():  
    .....          # body of function  
    .....
```

Even if a function is defined, it cannot be executed until it is called. Hence, for using a function, it must be called using *function call statement*. Calling of a function includes the function name and a list of arguments (no function body).

Syntax:

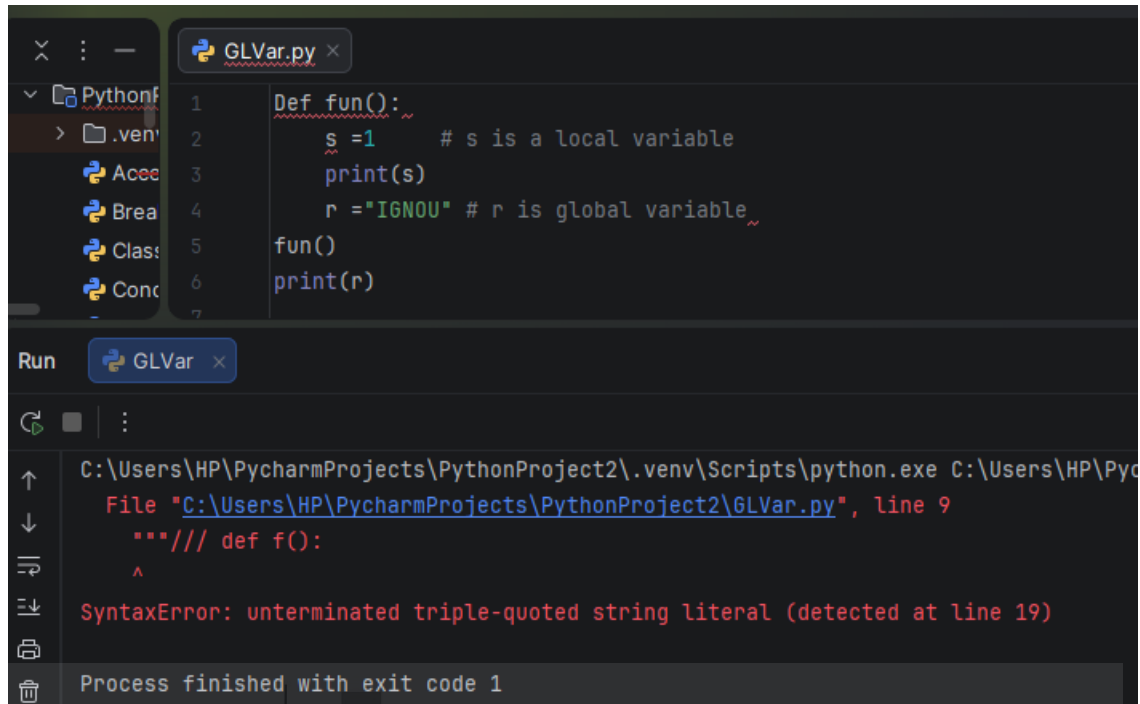
```
function_name( list of arguments)
```

2. The scope of a variable refers to the region of a program where that variable is recognized and accessible. Part of a code in which a variable can be accessed is called *the scope of a variable*.

Local variables: These are variables created within a function's body or function's scope. You cannot access them outside the function. Their scope is only limited to the function in which they are created.

Global variables: These are variables created outside any function. Their scope is global, hence can be used anywhere. Also, you can create global variables within a function using the global keyword.

Example:



```
1 def fun():
2     s = 1 # s is a local variable
3     print(s)
4     r = "IGNOU" # r is global variable
5     fun()
6     print(r)
```

Run GLVar

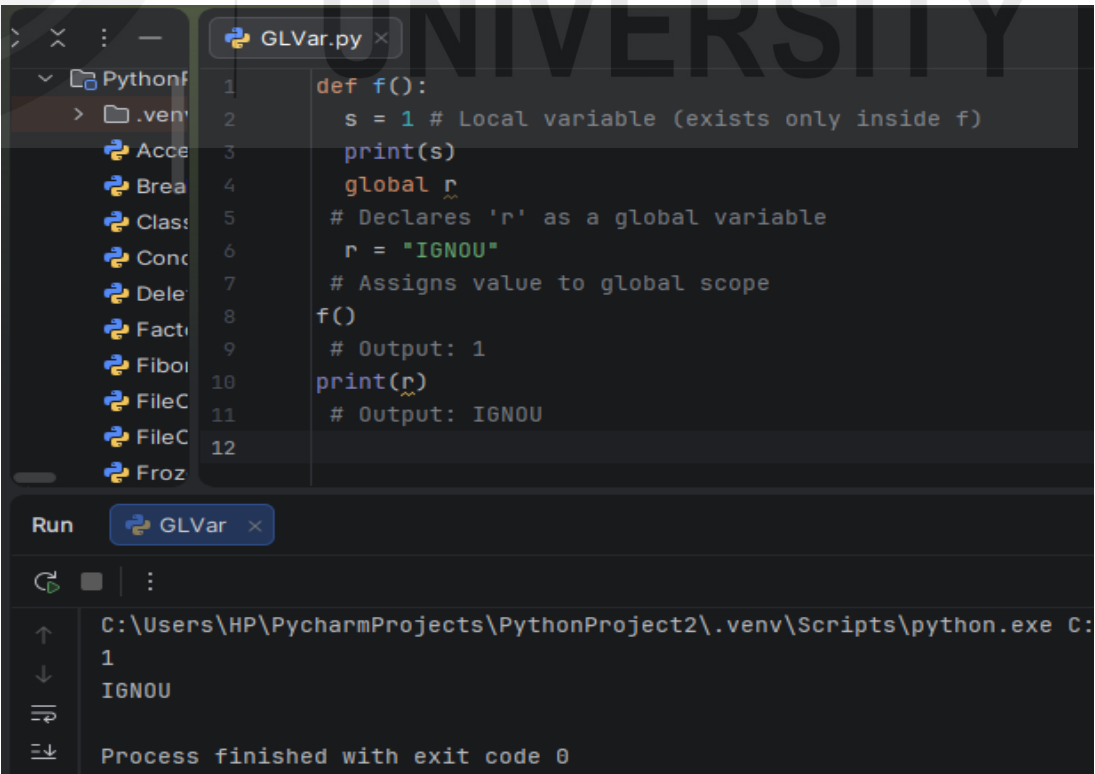
C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:\Users\HP\Pyd
File "C:\Users\HP\PycharmProjects\PythonProject2\GLVar.py", line 9
"""/// def f():
^
SyntaxError: unterminated triple-quoted string literal (detected at line 19)
Process finished with exit code 1

Code Correction and Explanation:

The global keyword is used inside a function to declare that a variable belongs to the global scope. It lets you create or modify a global variable from within the function.

In the code snippet above, r = "IGNOU" is a local variable because the global keyword was not used before assignment. Calling print(r) outside the function raised a NameError: name 'r' is not defined.

Below is the corrected code with proper Python syntax and declarations:



```
1 def f():
2     s = 1 # Local variable (exists only inside f)
3     print(s)
4     global r
5     # Declares 'r' as a global variable
6     r = "IGNOU"
7     # Assigns value to global scope
8     f()
9     # Output: 1
10    print(r)
11    # Output: IGNOU
12
```

Run GLVar

C:\Users\HP\PycharmProjects\PythonProject2\.venv\Scripts\python.exe C:
1
IGNOU
Process finished with exit code 0

- Without global r, r remains restricted to f() and is destroyed when f() finishes executing.
- Adding global r binds r to the global namespace, making it accessible outside the function call.

3.



```

1 def pattern():
2     for i in range(1, 6):
3         for j in range(1, i + 1):
4             print(j, end= " ")
5             print()
6 pattern()

```

Output:

```

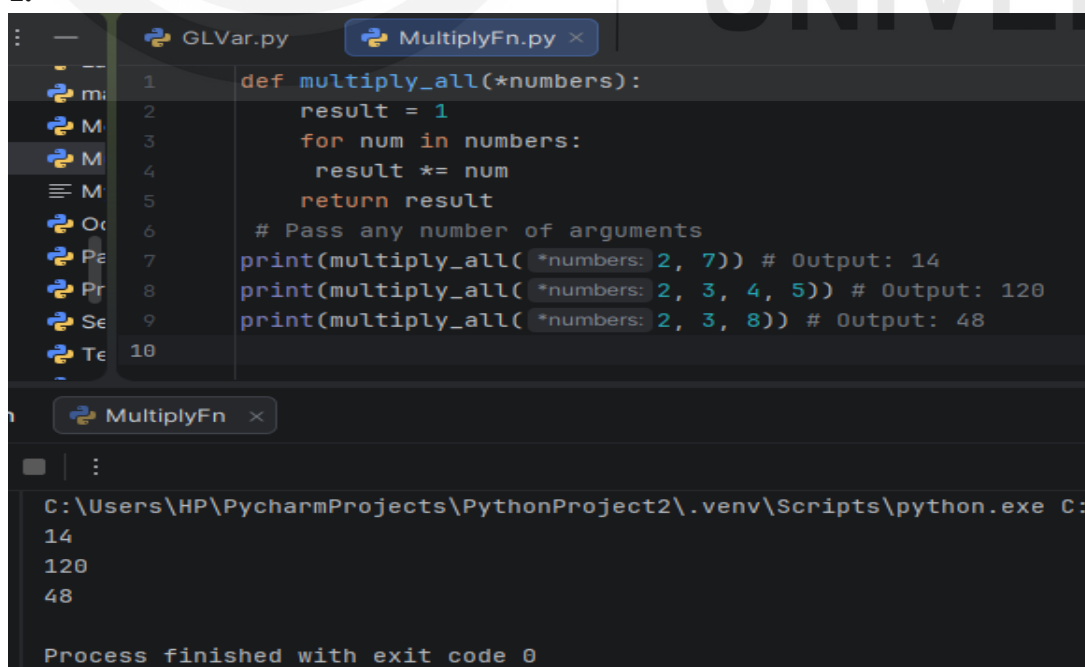
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5

```

Process finished with exit code 0

Answer to Check your Progress 2

1.



```

1 def multiply_all(*numbers):
2     result = 1
3     for num in numbers:
4         result *= num
5     return result
6 # Pass any number of arguments
7 print(multiply_all(*numbers: 2, 7)) # Output: 14
8 print(multiply_all(*numbers: 2, 3, 4, 5)) # Output: 120
9 print(multiply_all(*numbers: 2, 3, 8)) # Output: 48
10

```

Output:

```

14
120
48

```

Process finished with exit code 0

2. Python provides various mechanisms for passing arguments to a function.

1. Required arguments
2. Default arguments
3. Keyword arguments
4. Arbitrary arguments

3.

```
def unique(numbers):  
    out = []  
    for i in numbers:  
        if i not in out:  
            out.append(i)  
    return out  
  
ans=unique([1,2,1,1,2,2,3,4,6,3])  
print(ans)
```

4.

```
nums = [1, 2, 3, 4, 5]  
print("\nCub of numbers:")  
cube = list(map(lambda x: x ** 3, nums))  
print(cube)
```

5.

```
a = [1, 2, 3]  
b = [10, 20, 6]  
  
result = map(lambda x, y: x + y, a, b)  
print("\nAddition of two list:")  
print(list(result))
```

6.

```
def vowel(a):  
    v=['a','e','i','o','u']  
    if a in v:  
        return a  
  
l=['a','b','f','o','x','z','y']  
print("LIST OF VOWELS:")  
print(list(filter(vowel,l)))
```

Answer to Check your Progress 3

1.

```
from collections import Counter  
def wordcount(fname):  
    with open(fname) as f:  
        return Counter(f.read().split())  
  
print("Frequency of words:",wordcount("abc.txt"))
```

2.

```
n=int(input("enter number of lines:"))
c=1
file=open("test.txt","r")
for i in file:
    if c<=n:
        print(i)
        c+=1
file.close()
```

3.

```
def fsize(fname):
    import os
    status = os.stat(fname)
    return status.st_size

print("File size in bytes: ",fsize("test.txt"))
```

4.11 REFERENCES/FURTHER READINGS

1. Yashavant Kanetkar and Aditya Kanetkar, “Let Us Python”, 4th Edition, BPB Publications, New Delhi , 2022
2. Paul Deitel and Harvey Deitel , “Python For Programmers” , 1st Edition, Pearson India Education , Chennai, 2023
3. Sheetal Taneja and Naveen Kumar, “Python Programming a Modular Approach” , 1st Edition, Pearson India Education , Chennai , 2022
4. S. Shridhar, J. Indumathi and V.M. Hariharan , “Python Programming” , 1st Edition, Pearson India Education , Chennai, 2024
5. S. Suma Christal Mary , R. Josphine Leela, N. Senthamilarasi and Y. Chrisbin Jeeva , “Problem Solving and Python Programming”, 1st Edition, Ane Books , New Delhi, 2018